LeanCoder

The coding agent that proves every token

Build a Coding Agent That Gets the Job Done
on the Fewest Tokens

Global AI Hackathon, Bifrost cash track
Model: ollama/qwen2.5-coder:3b, local, through Bifrost v2.5.0
github.com/pakmultilinks-dot/bifrost-leancoder

Problem

AI coding tools send the whole repository on every turn. Teams pay for context they never needed, and nobody can prove what a task actually cost.

- A run can keep going after a provider outage
- A run can blow past a spending limit
- Private code can leak to a remote model
- Files can be written and commands run with no approval

Solution

LeanCoder sends only the code a task needs, found by a small local code search, and proves the saving with a measured token ledger.

- Lean mode: only the files the task needs go to the model
- Approval before every file write, shell command, Git action
- Per-task token budget with a hard stop
- Every call through Bifrost, local model fallback
- Private tasks run only on a local model

How lean mode works

1. Local code search finds the files the task needs
2. Only those files go to the model, nothing else
3. Files are re-read on every step, so a retry sees
   the current file state after its own patch
4. The model proposes a patch, the approval gate shows
   the diff, then the patch is applied and tests run
5. At most 5 steps, then the run stops and reports

Direct mode, the baseline, sends the whole repo context on every model call.

Architecture

- Agent loop: plan, read, propose patch, approve, apply, run tests, retry, report
- Tools: sandboxed file write, shell, and Git, all inside the task repo only
- Retrieval: small local code search, no vector DB
- Budget: per-task token and cost budget, hard stop
- Bifrost client: OpenAI-compatible, timeout, retry once on timeout only, clear error on 401
- Ledger and replay: every token counted from the model response, every run replayable offline

Python, requests only. Model: ollama/qwen2.5-coder:3b.

Measured results: 40 live runs

5 tasks, lean mode vs direct mode, same model, ollama/qwen2.5-coder:3b through Bifrost v2.5.0.

- Tokens per passing task, combined: lean 734.3, direct 842.5, about 13 percent fewer for lean
- Passes, combined: lean 15 of 20, direct 17 of 20
- Batch 3: lean 560.5 vs direct 579.75 per passing task, 4 of 5 each
- Batch 4: lean 797.5 vs direct 923.4, about 14 percent fewer, lean 11 of 15, direct 13 of 15
- Honest limit: lean had the lower mean on only 2 of 5 tasks in Batch 4, task4 lean ranged from 462 to 2652 tokens, no mean is a stable truth
- Cost is 0.0 for every run: the model runs locally

Outage drill and private-task proof

Outage drill: the client was pointed at a dead gateway.

- Result: plain error, 0 tokens, no invented success
- Error: Could not reach Bifrost at
  http://127.0.0.1:9999. Start Bifrost first, then retry.

Private-task proof: a task marked PRIVATE, run twice.

- Remote model: refused before any model call,
  0 model calls, 0 tokens
- Local model: it ran, retry passed in 2 steps,
  854 tokens

Judge verification

python3 verify.py

- Offline: no key, no Docker, no Bifrost
- Runs the full unit test suite: 83 tests passed
- Replays all 40 recorded traces: 40 of 40 matched
- Patches re-applied, hidden tests re-run, ledger token totals checked
- Result: PASS, exit 0, 4.8 seconds
- Failed tasks replay as failures too: replay checks what happened, it does not turn failures into passes

Public repo: github.com/pakmultilinks-dot/bifrost-leancoder

What is next

- A bigger model, when the machine allows it: a 7b model did not fit this 8 GB machine
- More tasks and more repeats, to shrink the noise in the means
- A remote free tier model as primary through Bifrost, with the local model as fallback, once a key exists
- Private code stays local in every case

The ledger stays honest either way: every run is measured, every failure is shown as a failure.

LeanCoder

Lean used about 13 percent fewer tokens per passing task across 40 live runs: 734.3 against 842.5, and passed 15 of 20 against 17 of 20.

Every token is counted. Every failure is shown. Verification needs no key.

github.com/pakmultilinks-dot/bifrost-leancoder

A place in the Bifrost cash track.